

PROJECT REPORT

LEDGER

A Web-Based Personal Finance & Expense Tracking Application

Project URL	https://ledger67.vercel.app
Technology Domain	Full-Stack Web Development
Frontend	React.js, Tailwind CSS
Backend & Database	Firebase, Firestore
Deployment	Vercel
Project Type	1st Year CSE Minor Project
Submitted To	Department of CSE

Submitted by: Team Ledger

ledger67.vercel.app

Table of Contents

1. Introduction	3
2. Problem Statement	3
3. Objectives of the Project	4
4. System Overview & Architecture	5
5. Technology Stack	6
6. Core Features & Modules	8
7. User Interface Design	10
8. Database Design	11
9. Authentication & Security	12
10. Implementation Details	13
11. Testing & Validation	15
12. Results & Outcomes	17
13. Future Scope	18
14. Conclusion	19
15. References	20

1. Introduction

Ledger is a web-based personal finance management application designed to simplify the way individuals — particularly college students — track their daily income and expenditure. Built as a first-year Computer Science Engineering project, Ledger addresses a practical and widely felt need: the absence of a lightweight, easy-to-use, and visually intuitive tool for managing personal budgets in real time.

The application is hosted publicly at <https://ledger67.vercel.app> and can be accessed from any device with an internet connection. It offers a modern dark-mode interface, secure user authentication, instant transaction logging, category-based classification, and visual analytics — all without requiring any installation or technical expertise from the end user.

This report documents the complete development lifecycle of Ledger — from problem identification and system design to implementation, testing, and deployment. It reflects the practical application of concepts studied in the first year of the B.Tech (CSE) programme, including web technologies, database management, UI/UX design, and software engineering principles.

1.1 Background

Financial literacy and disciplined budgeting are skills that significantly influence a person's quality of life. However, surveys consistently show that a majority of college students lack structured financial habits. The primary reason cited is not unwillingness, but the absence of a tool that is fast, frictionless, and visually rewarding to use. Ledger was conceived to fill exactly this gap — offering a digital alternative to spreadsheets and paper ledgers that is purpose-built for the digital-native student generation.

1.2 Scope

The scope of this project encompasses the design, development, testing, and deployment of a full-stack web application. The project covers frontend development using React.js, backend operations via Firebase, data persistence using Firestore, and cloud hosting on the Vercel platform. The application is intended for personal use by individuals wishing to maintain a clear record of their financial activities.

2. Problem Statement

Despite the widespread availability of smartphones and internet connectivity, a significant proportion of college students continue to struggle with financial planning. The core issues can be summarised as follows:

- **Absence of real-time financial awareness:** Students are often unaware of their current balance or spending rate until they check their bank statement at month-end, by which point overspending has already occurred.
- **Inefficiency of manual methods:** Maintaining a physical notebook or generic spreadsheet for expense tracking is time-consuming and prone to errors and omissions.
- **Lack of categorisation:** Without structured categorisation of transactions (e.g., food, travel, utilities), it is difficult to identify areas of excessive spending.
- **No visual representation:** Raw numbers in a ledger or spreadsheet do not provide the intuitive understanding that a chart or graphical summary can offer.
- **Privacy concerns:** Generic finance apps often request excessive permissions or store data on third-party servers with unclear privacy policies.

These challenges collectively result in poor financial decision-making, last-minute budget crunches, and unnecessary financial stress among students. Ledger was developed as a targeted solution to each of these pain points, providing a secure, intuitive, and visually rich platform for personal expense management.

Statistic	Finding	Source / Basis
Students without budgets	~73% lack a consistent habit	Internal survey (2025)
Common overspend margin	Up to 2x planned budget	Peer review data
Preference for digital tools	Over 90% prefer apps to paper	Questionnaire, N=60

Table 2.1 — Survey statistics motivating the Ledger project

3. Objectives of the Project

The following objectives guided the design and development of the Ledger application:

1. Develop a fully functional full-stack web application that enables authenticated users to record, view, and manage personal financial transactions.
2. Implement a clean, responsive, and mobile-friendly user interface that works seamlessly across devices of varying screen sizes.
3. Provide a real-time dashboard that displays the user's current balance, total income, and total expenses at a glance.
4. Enable category-based classification of transactions to support granular spending analysis.
5. Integrate interactive charts and visual summaries that help users interpret their financial patterns without requiring data literacy.
6. Ensure user data privacy through secure authentication and per-user data isolation in the database.
7. Deploy the application to a cloud platform so that it is publicly accessible, scalable, and available without downtime.
8. Follow industry-standard coding practices, component-based architecture, and version control workflows throughout development.

These objectives were evaluated against the final deliverable and used as the basis for functional testing. Each objective maps to one or more testable features described in Chapter 11.

4. System Overview & Architecture

Ledger follows a modern three-tier web application architecture, separating concerns across the presentation layer, the application logic layer, and the data persistence layer. This separation enhances maintainability, scalability, and testability.

4.1 Architecture Overview

The system consists of three principal layers that communicate through well-defined interfaces:

- **Presentation Layer (Client):** Built with React.js and styled using Tailwind CSS. The client renders the user interface, manages component state, and communicates with Firebase directly via the Firebase Web SDK.
- **Application Logic Layer:** Handled within React components and Firebase Cloud Functions (where applicable). Business rules such as balance calculation, category aggregation, and input validation are applied at this layer.
- **Data Persistence Layer:** Powered by Google Firebase, specifically Firestore (a NoSQL document database) for structured transaction data and Firebase Authentication for identity management.

4.2 System Architecture Diagram

The data flow within Ledger proceeds as follows: the user interacts with the React.js frontend, which communicates with Firebase Authentication to verify identity. Once authenticated, the application reads and writes to Firestore, with changes reflected in real time on the client via Firestore's live data listeners.



Figure 4.1 — Three-tier architecture of the Ledger application

4.3 Deployment Architecture

The compiled React application is deployed on Vercel, a cloud platform that provides global content delivery, automatic HTTPS, and CI/CD integration with GitHub. Every push to the main branch triggers an automated build and deployment pipeline, ensuring that the live application always reflects the latest stable version of the codebase.

5. Technology Stack

The selection of technologies for Ledger was guided by three criteria: developer familiarity (appropriate for a first-year project), community support and documentation quality, and suitability for the requirements of a real-time, data-driven web application.

5.1 Frontend Technologies

5.1.1 React.js

React.js is an open-source JavaScript library developed and maintained by Meta (formerly Facebook). It employs a component-based architecture in which the user interface is broken into reusable, self-contained units called components. Each component manages its own state and renders a portion of the UI. React uses a virtual DOM to efficiently update only the portions of the page that have changed, resulting in fast and smooth interactions. For Ledger, React enabled modular development of the dashboard, transaction form, category filter, and analytics panels.

5.1.2 Tailwind CSS

Tailwind CSS is a utility-first CSS framework that provides a comprehensive set of low-level CSS classes. Rather than writing custom stylesheets, developers compose styles directly within JSX markup using class names. This approach accelerates development significantly and results in highly consistent, responsive layouts. Tailwind's dark mode support was particularly valuable for implementing Ledger's signature dark-mode aesthetic without additional configuration.

5.2 Backend & Database Technologies

5.2.1 Firebase Authentication

Firebase Authentication, a service within Google's Firebase platform, provides a complete identity solution supporting email/password login, Google OAuth, and several other providers. It handles token generation, session management, and security entirely server-side, eliminating the need for Ledger to store or manage passwords. JWT (JSON Web Tokens) are issued automatically and verified on each request.

5.2.2 Cloud Firestore

Cloud Firestore is a flexible, scalable NoSQL cloud database designed for mobile and web applications. Data is organised into collections and documents (analogous to tables and rows in a relational database, but more flexible). Firestore's real-time listeners push updates to connected clients instantly whenever the underlying data changes — a critical feature for Ledger's live

dashboard. Its client-side SDK integrates directly with React, removing the need for a separate backend server.

5.3 DevOps & Tooling

Tool	Category	Version	Purpose
React.js	Frontend	18.x	UI component library
Tailwind CSS	Styling	3.x	Utility-first CSS framework
Firebase	BaaS	9.x (modular)	Auth & Firestore database
Vercel	Hosting/CDN	Latest	Deployment & CI/CD
Git + GitHub	Version Control	2.x	Code versioning & collaboration
VS Code	IDE	Latest	Development environment
npm / Node.js	Package Manager	18.x LTS	Dependency management

Table 5.1 — Technology stack summary

6. Core Features & Modules

Ledger is structured around eight primary feature modules, each serving a distinct user need. The following sections describe each module in detail.

6.1 User Authentication Module

The authentication module provides secure login and registration functionality. Users may sign in using a Google account (via OAuth 2.0) or via email and password. Upon successful authentication, Firebase issues a JWT that is transparently attached to all subsequent Firestore requests. Session persistence is handled client-side, so users remain logged in across browser sessions until they explicitly sign out.

6.2 Transaction Management Module

This is the core functional module of Ledger. It enables users to:

- Add a new transaction by specifying the amount, type (income or expense), category, date, and an optional note.
- View the complete transaction history in reverse chronological order.
- Delete transactions that were entered incorrectly.
- Filter the transaction list by month to review a specific period's activity.

Each transaction is stored as a document within the user's transactions sub-collection in Firestore. The data model ensures complete isolation between users — no user can read or write another user's data.

6.3 Dashboard & Balance Summary Module

The dashboard presents three key metrics prominently at the top of the screen: total balance (income minus expenses), total income, and total expenses for the selected period. These figures update in real time as transactions are added or removed. The dashboard provides immediate financial awareness without requiring any navigation or interaction from the user.

6.4 Category Classification Module

Transactions are tagged with predefined categories (e.g., Food, Travel, Bills, Entertainment, Healthcare, Shopping, Other). Categories serve two purposes: they make the transaction list easier to scan, and they feed the analytics module's breakdown charts. Users may also create custom categories to suit their individual spending patterns.

6.5 Analytics & Reporting Module

The analytics module visualises spending data through interactive charts:

- **Pie Chart:** Displays the proportional breakdown of expenses by category, allowing users to identify their dominant spending areas at a glance.
- **Bar Chart:** Shows a month-by-month comparison of income versus expenses, revealing trends and seasonal patterns.
- **Summary Cards:** Highlight the highest spend category, average monthly saving, and largest single transaction.

Charts are rendered using a JavaScript charting library and are fully interactive — hovering over segments displays exact values and percentages.

6.6 Monthly Filter Module

Users can navigate between months using a date picker or navigation arrows. Selecting a different month instantly reloads the transaction list, dashboard totals, and charts to reflect only that month's data. This module is particularly useful for comparing spending habits across different months or reviewing historical data.

6.7 Responsive UI Module

Ledger is designed with a mobile-first philosophy. The layout adapts fluidly to screens of all sizes — from 320px (small smartphones) to full desktop monitors. Navigation elements collapse into a hamburger menu on small screens, and the dashboard cards stack vertically to prevent horizontal scrolling. The dark-mode colour scheme reduces eye strain during extended use.

6.8 Data Export Module

Users may export their transaction history as a structured data file for external use — such as importing into a spreadsheet application for further analysis or maintaining personal records. This feature reinforces transparency and gives users full ownership of their data.

7. User Interface Design

The user interface of Ledger was designed with two guiding principles: clarity and efficiency. Every interaction should require the minimum number of steps, and every screen should present information in an immediately understandable form.

7.1 Design Philosophy

- **Dark Mode First:** The entire application uses a dark-mode colour scheme with a deep navy background (#0F172A), slate panels (#1E293B), and cyan accent (#22D3EE). This reduces eye fatigue, particularly for users who review finances in the evening.
- **Minimalist Layout:** Each screen is focused on a single task. The dashboard, transaction form, and analytics are separate views to avoid cognitive overload.
- **Consistent Typography:** A single sans-serif typeface is used throughout with a clear size hierarchy: 32px for primary headings, 20px for section titles, and 14px for body text.
- **Colour-Coded Feedback:** Income figures are displayed in green, expenses in red, and the net balance in cyan — providing instant visual cues without requiring the user to read labels.

7.2 Key Screens

Screen	Description
Login / Register	Clean authentication screen with Google sign-in and email/password options. Minimal fields to reduce friction.
Dashboard	Three summary cards (Balance, Income, Expenses) followed by the recent transaction list. Month navigator at the top.
Add Transaction	A modal or dedicated page with fields for amount, type, category, date, and note. Single-tap save.
Analytics	Full-screen charts view with pie chart (category breakdown) and bar chart (monthly comparison).
Transaction History	Searchable, filterable list of all transactions with colour-coded type indicators and delete option.

Table 7.1 — Key screens and their purpose

7.3 Accessibility Considerations

Ledger's interface was designed with basic accessibility in mind. All interactive elements meet the WCAG 2.1 minimum contrast ratio of 4.5:1 for normal text. Form inputs are labelled explicitly to support screen readers. The tab order follows a logical top-to-bottom, left-to-right sequence, enabling keyboard-only navigation.

8. Database Design

Ledger uses Cloud Firestore as its primary data store. Firestore organises data hierarchically into collections and documents. The Ledger data model consists of two primary collections: users and transactions.

8.1 Data Model

8.1.1 users Collection

Each authenticated user is represented by a document in the users collection. The document ID corresponds to the user's unique Firebase Authentication UID, ensuring a direct and secure mapping between identity and data.

Field Name	Data Type	Description
uid	string	Firebase Authentication unique user ID
name	string	User's display name
email	string	User's email address
createdAt	timestamp	Account creation timestamp
photoURL	string (optional)	Profile picture URL from OAuth provider

Table 8.1 — users collection schema

8.1.2 transactions Sub-Collection

Each user document contains a transactions sub-collection. This hierarchical structure ensures that a user can only access their own transactions through Firestore Security Rules.

Field Name	Data Type	Description
id	string	Auto-generated Firestore document ID
amount	number	Transaction value in INR
type	string	'income' or 'expense'
category	string	Predefined or custom category label
date	timestamp	Date of the transaction
note	string (optional)	User-provided description or note
createdAt	timestamp	Server-side creation timestamp

Table 8.2 — transactions sub-collection schema

8.2 Firestore Security Rules

Firestore Security Rules enforce data access policies at the database level, independent of the application code. The rules for Ledger are configured as follows:

- A user may only read, write, update, or delete documents within their own `users/{uid}` path.
- All requests must be authenticated; unauthenticated requests are denied by default.
- The transaction sub-collection is only accessible to the parent document's owner.

These rules ensure that even if a malicious actor bypassed the application's frontend, they could not access another user's data directly through the Firebase API.

9. Authentication & Security

9.1 Authentication Flow

Ledger supports two authentication methods:

- 9. Email and Password: The user provides their email address and a password. Firebase Authentication hashes and salts the password server-side; it is never stored in plaintext anywhere in the system.
- 10. Google OAuth 2.0: The user clicks 'Sign in with Google', which opens a secure consent screen managed entirely by Google. Upon approval, Firebase receives an identity token, creates a user record if one does not exist, and establishes a session.

In both cases, Firebase issues a short-lived JWT (valid for one hour) that is automatically refreshed by the Firebase SDK. This token is included in all Firestore requests and verified server-side before data is returned.

9.2 Security Measures

Security Measure	Implementation Details
Password Hashing	Managed entirely by Firebase Auth; bcrypt-equivalent hashing with salt
JWT Token Auth	Short-lived tokens (1 hr) with automatic refresh via Firebase SDK
Firestore Rules	Server-side per-user data isolation; unauthenticated requests rejected
HTTPS Enforcement	Vercel enforces TLS 1.2+ on all traffic; plain HTTP redirected
No Password Storage	Application never stores, logs, or transmits user passwords
Input Validation	Client-side validation on all form fields; malformed data rejected
XSS Prevention	React's JSX escapes all rendered values by default

Table 9.1 — Security measures implemented in Ledger

10. Implementation Details

This chapter describes the development process, project structure, and key implementation decisions made during the construction of Ledger.

10.1 Project Structure

The Ledger codebase follows a standard React project layout with clear separation between components, pages, services, and utility functions:

- `src/components/` — Reusable UI components (TransactionCard, CategoryBadge, SummaryCard, Chart, etc.)
- `src/pages/` — Top-level route components (Dashboard, Login, Analytics, TransactionHistory)
- `src/services/` — Firebase initialisation and Firestore data access functions (addTransaction, getTransactions, deleteTransaction)
- `src/hooks/` — Custom React hooks (useAuth, useTransactions) for state management and data fetching
- `src/utils/` — Helper functions for date formatting, currency formatting, and category mapping

10.2 State Management

Ledger uses React's built-in Context API combined with custom hooks for state management. The AuthContext provides authentication state (current user, loading status, sign-in/sign-out functions) to all components without prop drilling. The TransactionContext aggregates transaction data, computed totals, and filter state, ensuring consistent data across the dashboard, history list, and analytics views.

This approach was chosen over third-party state management libraries (such as Redux) to keep the codebase lean and appropriate for a first-year project, while still demonstrating solid understanding of state management patterns.

10.3 Real-Time Data Synchronisation

Firestore's `onSnapshot()` listener is used to subscribe to the user's transactions collection. This listener fires immediately with the current data and then again whenever a document is added, modified, or deleted. The React component updates its state in response, causing the UI to re-render with the latest data. This mechanism eliminates the need for periodic polling and ensures the dashboard always reflects the true current state of the user's finances.

10.4 Development Workflow

The project was developed following an iterative development model, proceeding through the following phases:

11. Requirement Analysis: Defined user stories, feature list, and non-functional requirements.
12. System Design: Designed the data model, component hierarchy, and application flow diagrams.
13. Frontend Development: Built and tested UI components in isolation using mock data.
14. Backend Integration: Connected Firebase Authentication and Firestore; replaced mock data with live database calls.
15. Testing: Conducted functional, usability, and performance testing (see Chapter 11).
16. Deployment: Pushed the production build to Vercel; configured custom domain and environment variables.
17. Review & Documentation: Prepared this report and the presentation deck.

10.5 Version Control & Collaboration

The entire codebase is managed using Git with a remote repository hosted on GitHub. Branches were used for feature development, and pull requests were reviewed before merging into the main branch. Commit messages follow the Conventional Commits specification (e.g., feat:, fix:, docs:) to maintain a readable project history.

11. Testing & Validation

A structured testing programme was conducted to verify that Ledger meets all defined functional and non-functional requirements. Testing was carried out across three dimensions: functional correctness, usability, and performance.

11.1 Functional Testing

Functional tests verified that each module behaves as specified under normal conditions and edge cases. The following test cases were executed:

#	Test Case	Expected Result	Status	Notes
TC -01	Register with valid email & password	Account created, redirected to dashboard	Pass	
TC -02	Register with existing email	Error message displayed	Pass	
TC -03	Sign in with Google OAuth	Authenticated, dashboard loaded	Pass	
TC -04	Add income transaction	Balance increases, entry in list	Pass	
TC -05	Add expense transaction	Balance decreases, entry in list	Pass	
TC -06	Add transaction with empty amount	Validation error shown	Pass	
TC -07	Delete a transaction	Entry removed, balance updated	Pass	
TC -08	Filter by month (no transactions)	Empty state message shown	Pass	
TC -09	Filter by month (with transactions)	Correct entries displayed	Pass	
TC -10	Dashboard totals accuracy	Totals match sum of transactions	Pass	Verified for 50 entries
TC -11	Pie chart renders on analytics	Correct category percentages	Pass	
TC -12	Bar chart renders monthly data	Correct income/expense bars	Pass	
TC -13	Session persistence after refresh	User remains logged in	Pass	
TC -14	Sign out	Session cleared, redirected to login	Pass	
TC -15	Access dashboard while logged out	Redirected to login page	Pass	Route guard working

Table 11.1 — Functional test cases and results

11.2 Usability Testing

Usability testing was conducted with a group of 10 first-year students who had not previously seen the application. Participants were asked to complete five tasks without guidance: register an account, add three transactions, check their monthly balance, view the analytics chart, and sign out. The following results were recorded:

- Average task completion rate: 96%
- Average time to add a transaction: 28 seconds
- All 10 participants successfully completed the sign-in and sign-out tasks without assistance
- 8 of 10 participants rated the interface as 'intuitive' or 'very intuitive'
- Primary improvement suggestion: Add a search bar for the transaction list (noted for future development)

11.3 Performance Testing

Performance was assessed using Google Lighthouse and manual browser profiling:

Metric	Result	Benchmark
Lighthouse Performance Score	91 / 100	> 90 (Good)
First Contentful Paint	1.2 s	< 1.8 s
Largest Contentful Paint	1.9 s	< 2.5 s
Time to Interactive	2.1 s	< 3.8 s
Firestore Read Latency	< 180 ms	< 500 ms

Table 11.2 — Performance benchmarks

12. Results & Outcomes

The Ledger application was successfully developed, deployed, and verified against all stated objectives. This chapter summarises the key outcomes of the project.

12.1 Objective Achievement Summary

#	Objective	Status	Evidence
O1	Full-stack web application with CRUD operations	Achieved	Live app
O2	Responsive, mobile-friendly interface	Achieved	Tested on 5 devices
O3	Real-time balance dashboard	Achieved	onSnapshot listener
O4	Category-based classification	Achieved	8 default categories
O5	Interactive charts and analytics	Achieved	Pie & bar charts
O6	Secure auth & data isolation	Achieved	Firestore rules
O7	Cloud deployment (zero downtime)	Achieved	Vercel (99.9% uptime)
O8	Standard coding & version control	Achieved	GitHub + Conventional Commits

Table 12.1 — Objective achievement summary

12.2 Key Quantitative Outcomes

- 15 functional test cases executed with a 100% pass rate
- Lighthouse performance score of 91/100 on the deployed application
- 96% task completion rate in usability testing with first-time users
- Application live and publicly accessible at ledger67.vercel.app
- Codebase comprises approximately 2,400 lines of source code across 38 files
- Average Firestore read latency of under 180 ms from the Indian subcontinent

12.3 Learning Outcomes

Beyond the technical deliverable, the project yielded significant learning outcomes for the development team:

- Practical experience with React.js component architecture, hooks, and context API
- Hands-on implementation of Firebase Authentication and Firestore in a production environment
- Understanding of NoSQL data modelling and Firestore security rules
- Experience with the full software development lifecycle — from requirements to deployment

- Exposure to cloud deployment workflows and CI/CD pipelines via Vercel and GitHub

13. Future Scope

While the current version of Ledger successfully meets its defined objectives, several enhancements have been identified for future development phases. These enhancements are grouped by priority and development effort.

13.1 Short-Term Enhancements (Phase 2)

- **Budget Goals:** Allow users to set monthly spending limits per category. The dashboard would display a visual indicator when the user approaches or exceeds a budget threshold.
- **Transaction Search:** Add a full-text search bar to the transaction history screen, enabling users to quickly locate specific transactions by description or amount.
- **Recurring Transactions:** Enable users to define recurring income or expense entries (e.g., monthly rent, weekly grocery budget) that are automatically logged at the specified interval.
- **PDF Export:** Generate a formatted monthly financial report as a downloadable PDF, suitable for record-keeping or submission to institutions.

13.2 Medium-Term Enhancements (Phase 3)

- **Native Mobile Application:** Develop dedicated Android and iOS applications using React Native, sharing the existing business logic and Firebase backend. Native apps would enable push notifications and offline-first functionality.
- **AI-Powered Expense Advisor:** Integrate a machine learning model (or a language model API) to analyse the user's spending patterns and generate personalised saving recommendations and anomaly alerts.
- **Multi-Currency Support:** Add support for international currencies with live exchange rate conversion, valuable for students studying or travelling abroad.

13.3 Long-Term Vision (Phase 4+)

- **UPI & Bank Integration:** Connect with India's UPI ecosystem and major banking APIs to automatically import transactions, eliminating manual entry entirely.
- **Group Expense Management:** Support shared expense tracking among groups (e.g., flatmates or project teams), with automatic calculation of individual dues and in-app settlement.
- **Financial Goal Tracking:** Allow users to define long-term savings goals (e.g., buying a laptop, funding a trip) and track progress through visual milestones.

14. Conclusion

Ledger represents a practical and complete solution to the challenge of personal finance management for college students. By combining a clean, mobile-first user interface with real-time data synchronisation, secure authentication, and visual analytics, the application transforms a traditionally tedious activity into an intuitive and even engaging daily habit.

The project successfully demonstrates the integration of modern frontend technologies (React.js, Tailwind CSS) with a powerful cloud-based backend (Firebase, Firestore) to produce a production-quality web application within the timeframe and resource constraints of a first-year undergraduate project. All eight stated objectives were achieved, and the application scored a Lighthouse performance rating of 91 out of 100 on the deployed production build.

From a pedagogical standpoint, the development of Ledger provided hands-on exposure to the complete software development lifecycle — from problem identification and requirement analysis through design, implementation, testing, and deployment. The project reinforced theoretical concepts covered in the first year of the B.Tech (CSE) programme and introduced practical skills in version control, cloud services, and agile development that will be valuable throughout the team's academic and professional careers.

The codebase and deployment infrastructure have been designed with extensibility in mind, providing a solid foundation for the enhancements outlined in the Future Scope chapter. Ledger is live, publicly accessible, and ready to serve as a practical financial tool for anyone who wishes to take control of their personal finances.

"Financial freedom begins with knowing where every rupee goes."

15. References

The following resources were consulted during the design, development, and documentation of the Ledger project:

18. React.js Official Documentation. (2025). Getting Started with React. Retrieved from <https://react.dev/learn>
19. Google Firebase. (2025). Firebase Documentation — Authentication and Firestore. Retrieved from <https://firebase.google.com/docs>
20. Tailwind Labs. (2025). Tailwind CSS Documentation. Retrieved from <https://tailwindcss.com/docs>
21. Vercel. (2025). Vercel Platform Documentation — Deploying React Applications. Retrieved from <https://vercel.com/docs>
22. Mozilla Developer Network. (2025). JavaScript Reference and Web API Documentation. Retrieved from <https://developer.mozilla.org>
23. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
24. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
25. Google Developers. (2025). Web Fundamentals — Performance and Progressive Web Apps. Retrieved from <https://developers.google.com/web>
26. Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.
27. W3C. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. Retrieved from <https://www.w3.org/TR/WCAG21/>

— End of Report —

Ledger | ledger67.vercel.app | CSE Department | 2025–26